Home
Overview
Quickstart
Libraries
Changelog
Troubleshooting
Release notes
Coding with Gemini
Models
Gemini 3
All models
Gemini 3 Pro
Gemini 3 Flash
Nano Banana Pro
Nano Banana
Veo 3.1
Veo 3
Veo 2
Imagen 4
Imagen 3
Gemini TTS
Gemini Live API
Gemini Embedding
Learn about
Prompting
Image generation
Video generation
Speech generation
Long context
Thinking
Document understanding
Vision understanding
Audio understanding
Live API
Function calling
Code execution
Grounding with Google Search
Grounding with Google Maps
URL context
Computer use
Files API
Text generation
Embeddings
Tokens
Tuning
Batch API
Safety
Models
Gemini API Cookbook
Gemini API Quickstart
Gemini API Starter Template
Gemini CLIP
Gemini Live API Tool Use
Gemini API Python SDK Cookbook
Gemini API Node.js SDK Cookbook
Gemini API Go SDK Cookbook
Gemini API Dart SDK Cookbook
Gemini API Swift SDK Cookbook
Gemini API Android SDK Cookbook
Gemini API Flutter SDK Cookbook
Gemini API Firebase SDK Cookbook
Gemini API Web SDK Cookbook
Gemini API Chrome SDK Cookbook
Gemini API Cloud SDK Cookbook
Gemini API Cloud Shell Tutorials
Gemini API Cloud Run Tutorials
Gemini API Firebase Tutorials
Gemini API Web Tutorials
Gemini API Chrome Tutorials
Gemini API Cloud Tutorials
Gemini API Cookbook Contribution Guidelines
Gemini API Cookbook Style Guide
Gemini API Cookbook Maintenance Guidelines
Gemini API Cookbook Contributor Agreement

Home Models

Gemini 3 Developer Guide

To fit our mission, Gemini 3 is frontier model for reasoning, multimodal understanding and agentic capabilities.

Gemini 3 is our most intelligent model family to date, built on a foundation of state-of-the-art reasoning. It's designed to bring ideas to life by handling complex tasks requiring deep contextual understanding across text, code, and multimodal inputs. This guide covers the features of the Gemini 3 model family and how to get the most out of them.

Gemini 3 Pro Gemini 3 Flash Nano Banana Pro Nano Banana Veo 3.1 Veo 3 Veo 2 Imagen 4 Imagen 3 Gemini TTS Gemini Live API Gemini Embedding

Explore our collection of Gemini 3 recipes to see how the model handles advanced reasoning, autonomous coding, and complex multimodal tasks.

Get started in Google AI Studio or read on for more details.

Meet the Gemini 3 series

Gemini 3 Pro is our flagship model, built for frontier performance across long-context processing, multimodal understanding, and agentic tasks.

Gemini 3 Flash is our latest frontier-class model, with the speed and intelligence at the scale and pricing of Flash.

Nano Banana Pro (also known as Gemini 3 Pro Image) is our highest-quality image generation model, and Nano Banana is our fast, high-volume, and low-latency variant. Veo 3.1 is our most capable video generation model, and Veo 3 is our stable, high-quality model.

Imagen 4 is our state-of-the-art model for text-to-image generation, and Imagen 3 is our previous generation model.

Gemini TTS is our speech generation model, and Gemini Live API is our real-time multimodal model.

Gemini Embedding is our state-of-the-art embedding model for text, image, and video.

New API features in Gemini 3

Gemini 3 introduces new parameters and API capabilities to give control over latency, cost, and multimodal fidelity.

Thinking control

Gemini 3 Pro is always on dynamic thinking by default to reason through problems. You can use the `thinking_level` parameter to control how much reasoning the model performs before it begins generating a response. Lower latency for simpler queries, or leave it unset to let the model dynamically reason over complex prompts.

Gemini 3 Flash also supports granular thinking budgets, which let you specify a token limit for the model to reason within, allowing you to balance latency, cost, and quality. Flash supports the same thinking levels listed below.

`thinking_level`	Gemini 3 Pro	Gemini 3 Flash	Gemini 3 Pro Image	Gemini 3 Flash Image	Description
default	dynamic (default)	dynamic (default)	dynamic (default)	dynamic (default)	Model dynamically reasons over the prompt to decide when and how deep to think. Balances reasoning quality and latency. Default if not specified.
`low`	Supported	Supported	Supported	Supported	Minimizes latency and cost. Suitable for simple instruction following, chat, or high-throughput applications.
`medium`	Supported	Supported	Supported	Supported	Balanced thinking for most tasks.
`high`	Supported (default)	Supported (default)	Supported (default)	Supported (default)	Maximizes reasoning depth. The model may take significantly longer to reach a first token, but the response is more carefully reasoned. Default if not specified.

```
from google import genai
from google.genai.types import GenerateContentConfig, ThinkingConfig, ThinkingLevel

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-pro-preview",
    contents="Find the race conditions in this multi-threaded C++ snippet: [illegible]",
    config=GenerateContentConfig(
        thinking_config=ThinkingConfig(thinking_level=ThinkingLevel.LOW)
    )
)

print(response.text)
```

```
import google.genai.types as types

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents="Find the race conditions in this multi-threaded C++ snippet: [illegible]",
    config=types.GenerateContentConfig(
        thinking_config=types.ThinkingConfig(thinking_budget=1024)
    )
)

print("\n\n------ Thoughts ------\n\n",
      response.candidates[0].content.parts[0].text,
      "\n\n------ Answer ------\n\n",
      response.candidates[0].content.parts[1].text)
```

```
import google.genai.types as types

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents="Find the race conditions in this multi-threaded C++ snippet: [illegible]",
    config=types.GenerateContentConfig(
        thinking_config=types.ThinkingConfig(thinking_budget=-1)
    )
)

print("\n\n------ Thoughts ------\n\n",
      response.candidates[0].content.parts[0].text,
      "\n\n------ Answer ------\n\n",
      response.candidates[0].content.parts[1].text)
```

Note: For complex reasoning, expect a slightly higher number of thinking tokens to leverage Gemini's full reasoning capabilities. Use API logs, OpenAI usage fields, or Google AI Studio to monitor the model's token usage. If you encounter latency issues in your application, you can reduce the thinking level to `low` or `medium` to constrain the model's reasoning.

```
import google.genai
from google.genai.types import GenerateContentConfig, ThinkingConfig, ThinkingLevel

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-pro-preview",
    contents="Find the race conditions in this multi-threaded C++ snippet: [illegible]",
    config=GenerateContentConfig(
        thinking_config=ThinkingConfig(thinking_level=ThinkingLevel.LOW)
    )
)

print(response.text)
```

Note: For complex reasoning, expect a slightly higher number of thinking tokens to leverage Gemini's full reasoning capabilities. Use API logs, OpenAI usage fields, or Google AI Studio to monitor the model's token usage. If you encounter latency issues in your application, you can reduce the thinking level to `low` or `medium` to constrain the model's reasoning.

Media resolution

Gemini 3 introduces granular control over multimodal vision processing via the `media_resolution` parameter. Higher resolutions improve the model's ability to read fine text or identify small details, but increase token usage and latency. The parameter can be set to `media_resolution_low`, `media_resolution_medium`, or `media_resolution_high` across Gemini 3 Pro and Gemini 3 Flash.

You can now set the resolution per individual media part in the API. Mixing resolutions across parts is supported for multi-image reasoning tasks, allowing you to allocate high-fidelity processing only for the parts of a prompt that require it.

`media_resolution`	Supported by	Recommended for	Token allocation
`media_resolution_low`	Gemini 3 Pro, Gemini 3 Flash	Chat, long-context tasks, or tasks where general scene understanding is sufficient.	280 tokens
`media_resolution_medium`	Gemini 3 Pro, Gemini 3 Flash	Standard vision tasks where a balance of quality and cost is desired.	560 tokens
`media_resolution_high`	Gemini 3 Pro, Gemini 3 Flash	Default for most vision tasks to ensure maximum quality.	1120 tokens
`media_resolution_ultra_high`	Gemini 3 Pro, Gemini 3 Flash	Required only when the model must read small text or dense details.	2240 tokens

Note: Gemini 3 models interpret compression artifacts in uploaded images, a common cause of lower quality in long-context or multi-image tasks. We recommend testing `media_resolution_high` for optimal performance in these cases.

```
from google import genai
from google.genai.types import GenerateContentConfig, Part, MediaResolution

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-pro-preview",
    contents=[
        "What is this brand?",
        Part(inline_data=dict(mime_type="image/jpeg", data=[illegible]),
             media_resolution=MediaResolution.MEDIA_RESOLUTION_LOW)
    ]
)

print(response.text)
```

Temperature

For all Gemini 3 models, we strongly recommend keeping the temperature parameter at its default value of `1.0`.

While previous models often benefited from tuning temperature to control creativity versus determinism, Gemini 3's reasoning capabilities are optimized for this default setting. Changing the temperature (setting it below `1.0`) may lead to unexpected behavior, such as looping or degraded performance, particularly in complex mathematical or reasoning tasks.

Thought signatures

Gemini 3 uses thought signatures to maintain reasoning context across API calls. These signatures are encrypted representations of the model's internal thought process. We recommend you pass these signatures back when the model makes function calls to the model in your request exactly as they were received.

Function calling: For strict validation, Gemini 3 enforces signatures on the first function call in the current turn. Missing signatures will result in a 400 error.

Note: Vertex AI enforces signature validation for all function calls to ensure higher quality and safety.

Text and streaming: For text or streaming responses, the API will return a signature in a final part. We recommend you include this signature in the next request to ensure optimal performance for the model.

Text and streaming: For text or streaming responses, the API will return a signature in a final part. We recommend you include this signature in the next request to ensure optimal performance for the model.

Image generation and editing: For Gemini 3 Pro Image, signatures are returned on the text parts of the response. For interleaved outputs (text + image), only the final part of the response will contain a signature.

The Gemini 3 Pro Image model also uses signatures to control conversation state for multi-turn image generation and editing. For more details, see the Gemini 3 Pro Image section below.

Rules of thumb: If you are manually modifying conversation history, or streaming function calls, you should expect the model to provide signatures in subsequent turns. If you are using standard text chat or single-turn function calling, you generally will not need to manage signatures.

For more information on how thought signatures work across all Gemini models, see our Thought signatures guide.

Structured Outputs with tools

Gemini 3 models allow you to combine Structured Outputs with built-in tools, including Grounding with Google Search, URL Context, and code execution.

```
import google.genai
from google.genai.types import GenerateContentConfig, GoogleSearch, Tool

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-pro-preview",
    contents="Search for all details for the latest Euro.",
    config=GenerateContentConfig(
        tools=[
            Tool(google_search=GoogleSearch()),
        ],
        response_mime_type="application/json",
        response_schema=list[QuickFinder],
    )
)

print(response.text)
```

Image generation

Gemini 3 Pro Image and Gemini 3 Flash Image offer advanced control over the creation and editing of visuals. Key capabilities include the ability to specify aspect ratios and resolutions, access real-time information via Google Search, and leverage the model's enhanced reasoning for complex creative tasks.

Image generation with Gemini: Gemini 3 Pro Image and Gemini 3 Flash Image are multimodal models that can generate and edit images conversationally. You can use natural language to specify aspect ratios, refine specific areas of an image, or ask the model to use Search to ground your creation in real-time data.

Gemini 3 Pro Image: This model is optimized for high-fidelity creative and professional tasks. It supports 2K and 4K resolutions and utilizes its reasoning capabilities for improved composition and text rendering.

Gemini 3 Flash Image: This model is optimized for speed and high-volume usage. It supports 1K resolution and is ideal for rapid iterations and high-throughput applications.

Thinking process: Both models use a thinking process by default to reason through the prompt. The API returns interim "thought" images to show the model's reasoning. Only the final generated image does not have a thinking process.

Thinking process: Both models use a thinking process by default to reason through the prompt. The API returns interim "thought" images to show the model's reasoning. Only the final generated image does not have a thinking process.

To enable Search grounding, configure `tools=[{"google_search": {}}]`. The model will automatically use web sources to ground its response.

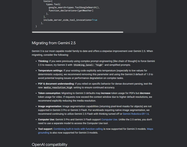
```
from google import genai
from google.genai.types import GenerateContentConfig, ImageConfig, Tool, GoogleSearch

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-pro-image-preview",
    contents="Visualize the current weather forecast for the next 5 days in San Francisco as a clean, modern chart.",
    config=GenerateContentConfig(
        tools=[{"google_search": {}}],
        image_config=ImageConfig(aspect_ratio="16:9",image_size="2K"),
        response_modalities=["TEXT","IMAGE"]
    )
)

for part in response.candidates[0].content.parts:
    if part.text:
        print(part.text)
    elif part.inline_data:
        image = part.as_image()
        image.save("weather_chart.png")
```

Example Response



Code Execution with Images

Gemini 3 can now write and execute code to manipulate and zoom in visual inputs, for instance, by cropping or rotating an image to better analyze the model's fundamental capabilities. Use cases include accurately counting objects, such as counting fingers, or editing images, such as cropping a photo to focus on a specific region of interest.

Example: Prompt: "Are these two paws the same size? Crop each paw and tell me the exact length of the longest toe on each."

Code execution and plotting: The model can now run bar charts, line charts, and scatter charts. Ask it to "Use code execution to plot the data you extracted" for precise data visualization.

Image processing: The model can perform standard image manipulations, such as calculating aspect ratios or converting color spaces, by writing and executing code.

To enable image thinking, configure `tools=[{"code_execution": {}}]`. The model will automatically use code to manipulate images and think with them.

```
from google import genai
from google.genai.types import GenerateContentConfig, Tool, CodeExecution

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents=[
        "Are these two paws the same size? Crop each paw and tell me the exact length of the longest toe on each.",
        types.Part.from_bytes(data=[illegible], mime_type="image/jpeg")
    ],
    config=GenerateContentConfig(
        tools=[Tool(code_execution=CodeExecution())]
    )
)

for part in response.candidates[0].content.parts:
    if part.text:
        print(part.text)
    elif part.executable_code:
        print("Generated code:\n", part.executable_code.code)
    elif part.code_execution_result:
        print("Code execution result:\n", part.code_execution_result.output)
    elif part.inline_data:
        print("Inline data: mime_type=", part.inline_data.mime_type)
```

For more details on code execution with images, see Code Execution.

Multimodal function responses

Gemini 3 extends our function calling capabilities to handle multimodal function responses, enabling the model to incorporate additional multimodal evidence during reasoning of the model. Standard function calling still supports text, image, and PDF formats.

```
import google.genai
from google.genai.types import GenerateContentConfig, Tool, FunctionDeclaration

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-pro-preview",
    contents="Analyze the image from the function call.",
    config=GenerateContentConfig(
        tools=[Tool(function_declarations=[{
            "name": "get_image_data",
            "description": "Retrieves an image based on a specified URL.",
            "parameters": {
                "type": "object",
                "properties": {
                    "url": {
                        "type": "string",
                        "description": "The URL of the image.",
                    },
                },
                "required": ["url"],
            },
        }])],
    )
)

print(response.text)
```

For models that support multimodal function responses, see Function calling.

Migrating from Gemini 2.5

Gemini 3 is our most intelligent model family and offers a massive improvement over Gemini 2.5. When migrating, consider the following:

- **Thinking:** If you were previously using complex prompt engineering (like chain of thought) to force Gemini 2.5 models to reason well, try setting `thinking_level` to `high` and simplify your prompts.
- **Temperature settings:** If your existing code explicitly specifies temperature, we recommend removing it and leveraging the Gemini 3 default setting of `1.0`. Changing temperature may affect reasoning capabilities in complex tasks, so we recommend using the default value for optimal performance.
- **Image quality:** If you relied on specific behaviors for image compression or quality, test the `media_resolution` options to see which best matches your requirements.
- **Token consumption:** Migrating to Gemini 3 may increase token usage for PDFs or dense documents. You can refer to our new prompting guide to understand how to optimize token usage.
- **Function calling:** Function calling now requires thought signatures to be returned in multi-turn conversations.
- **Text spelling:** Gemini 3 may find text spelling or character counting tasks challenging. Test these carefully if your application relies on these capabilities.
- **Grounding with Google Maps:** Not supported in Gemini 3 Flash with Grounding with Google Maps.
- **Code execution:** Code execution with Gemini 3 is now supported by Gemini 3 models. Image understanding is also supported by code execution.
- **Text-to-Speech:** Supported in Gemini 3 Pro and Flash.

OpenAI compatibility

If you are working with the OpenAI compatibility layer, standard parameters (`temperature`, `top_p`, `max_tokens`) and core instruction logic (`system_instruction`) are fully supported by Gemini 3. You can expect your existing prompts and logic to function as intended when migrating to our Gemini models.

Prompting best practices

Gemini 3 is a reasoning model, which changes how you should prompt.

- **Precise instructions:** Gemini 3's reasoning capabilities allow it to follow complex instructions. The model interprets its system instructions with high reasoning capacity to deliver high-quality responses. We recommend you provide clear and precise instructions to guide its final answer.
- **Concise prompts:** Unlike previous models, which may have required verbose prompt engineering, Gemini 3 prefers direct and concise communication. Instead of long, complex prompt engineering, you can now simply state the model's goal. The model can handle complex strategies internally.
- **Verbosity:** By default, Gemini 3 is less verbose than previous models. If your application requires more "chatty" or conversational responses, you must explicitly prompt the model to be more verbose.
- **Context management:** When providing large datasets (like long documents or videos), place your specific instructions or questions at the end of the prompt, after the data context. Instruct the model to reference the provided data before reasoning and answering its response.

For more advanced prompting techniques, see the DeepMind prompting guide.

FAQs

- **What is the knowledge cutoff for Gemini 3?** Gemini 3 models have a knowledge cutoff of January 2025. For recent events, Gemini 3 uses Search grounding.
- **What are the context window limits?** Gemini 3 models support a 1 million input context window and 64k output tokens.
- **Do you support the Batch API?** Yes, Gemini 3 supports the Batch API.
- **Is Context Caching supported?** Yes, Gemini 3 supports Context Caching.
- **Which tools are supported?** Gemini 3 supports Search grounding, Function calling, Code execution, and Structured outputs. Maps grounding is not supported in Gemini 3 Flash.
- **Is Gemini 3 supported in the Gemini CLI?** Yes, Gemini 3 supports the Gemini CLI.
- **Is Gemini 3 supported in Vertex AI?** Yes, Gemini 3 supports Vertex AI. You can find more details in the Vertex AI Gemini 3 guide.
- **Will Gemini 3 process my data for training?** Paid API calls for Gemini 3 Pro and Gemini 3 Flash Preview do not train models. Unpaid API calls may be used for training. For more details, see the Gemini API terms.
- **Which pricing is in effect for Gemini 3?** Pricing is based on token usage. For complete pricing details, see the Gemini API pricing page.

Next steps

- Get started with the Gemini 3 Cookbook.
- Review the Gemini API cookbook guide on thinking levels and how to migrate from thinking budget to thinking levels.

Was this helpful?
Yes No

Send feedback
Except as noted, content on this page is licensed under the Creative Commons Attribution 4.0 License, and code samples are licensed under the Apache 2.0 License. For details, see the Google Developers Site Policies.